



Introdução ao Gerenciamento de Memória

Prof. Pedro Botelho

Universidade Federal do Ceará

5 de maio de 2026



Armazenamento nos Computadores

Gerenciamento de Memória sem abstração

Abstrações Simples de Memória

Gerenciamento de Memória Livre

Conclusão



Armazenamento nos Computadores

Gerenciamento de Memória sem abstração

Abstrações Simples de Memória

Gerenciamento de Memória Livre

Conclusão

A memória física



- ▶ Espaço de memória RAM do computador
- ▶ Grande sequência de bytes com **endereços** individuais
- ▶ Conteúdo:
 - ▶ Sistema operacional
 - ▶ Aplicações em execução
 - ▶ Buffers de entrada/saída
 - ▶ Áreas de transferência (DMA)
- ▶ Dividida em áreas com diferentes finalidades

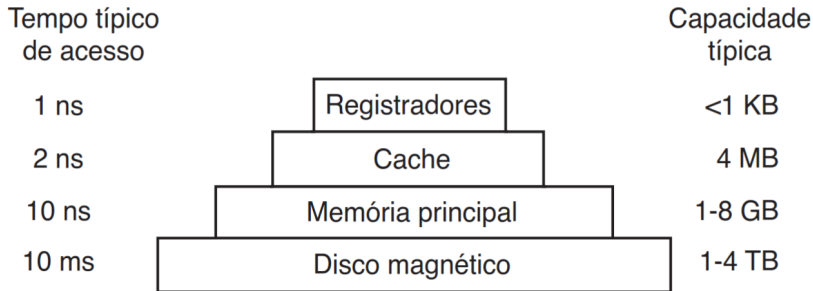


- ▶ Faixa de endereços que o processador consegue gerar
- ▶ Depende da arquitetura e tamanho de barramentos
 - ▶ 80386: CPU 32 bits, bus 32 bits, 2^{32} endereços
 - ▶ Core i7: CPU 64 bits, bus 48 bits, 2^{48} endereços
- ▶ **Não depende** da quantidade de RAM disponível!
 - ▶ Endereço **válido**: existe um byte de RAM naquela posição
 - ▶ Endereço **inválido**: não existe RAM naquela posição



- ▶ Crescimento constante da capacidade de memória
- ▶ Crescimento ainda maior da demanda pelos programas
- ▶ Mundo ideal em termos de memória
 - ▶ Capacidade infinita
 - ▶ Rápida
 - ▶ Não-volátil
 - ▶ Baixo custo
- ▶ Impossível de se obter
- ▶ Solução: Hierarquias de Memórias
 - ▶ Cache: muito rápida, alto custo e volátil (MB)
 - ▶ RAM: velocidade e custo médios e volátil (GB)
 - ▶ HD: velocidade e custo baixos e não volátil (TB)
- ▶ SO → Abstrai e gerencia essa hierarquia
- ▶ Gerenciador de memória → Uso/Disponível

Hiearquia de Memória



Características importantes:

- ▶ Capacidade
- ▶ Volatilidade
- ▶ Velocidade
- ▶ Custo por bit
- ▶ Consumo de energia



Armazenamento nos Computadores

Gerenciamento de Memória sem abstração

Abstrações Simples de Memória

Gerenciamento de Memória Livre

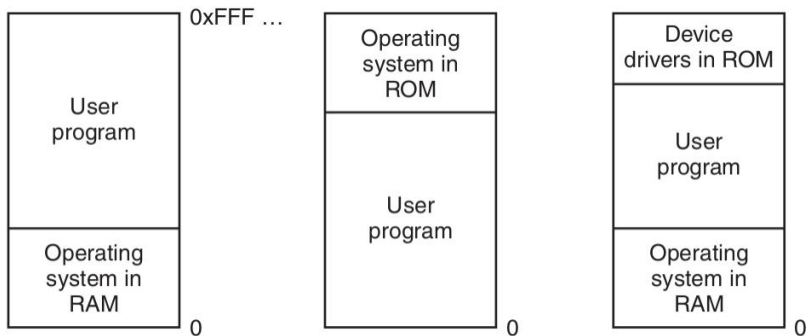
Conclusão

Gerenciamento sem abstração

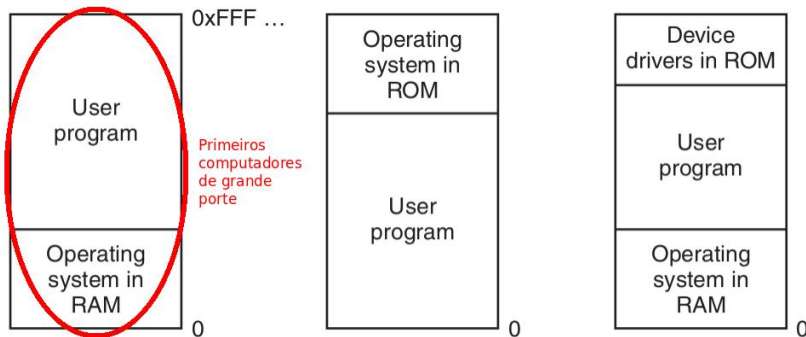


- ▶ Modelo de memória mais simples → Não ter abstração alguma
- ▶ Primeiros computadores → Zero abstração de memória
 - ▶ Modelo de memória = Memória física
- ▶ Ainda utilizado em sistemas simples → Sistemas Embarcados
- ▶ Conjunto de endereços
- ▶ Cada endereço → Célula com um número de bits

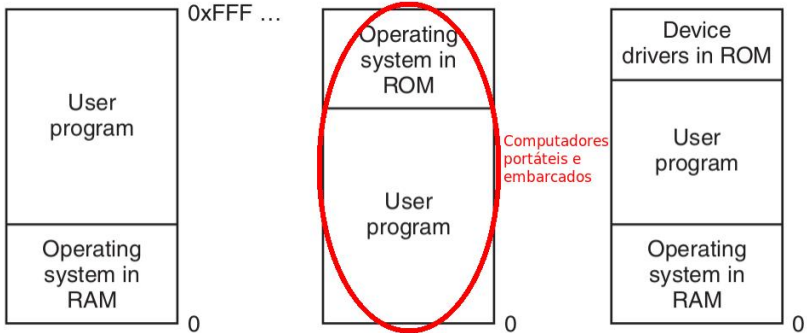
Exemplos de Modelos de Memória



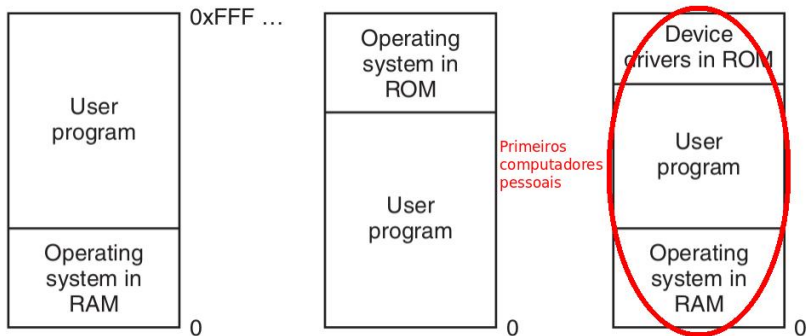
Exemplos de Modelos de Memória



Exemplos de Modelos de Memória



Exemplos de Modelos de Memória



Multiprogramação nesse cenário



- ▶ Apenas um processo por vez
- ▶ SO copia programa do disco para memória e executa
- ▶ Quando execução termina, SO pode executar outro processo
- ▶ Ao executar outro processo, memória é sobrescrita
- ▶ Possibilidade teórica de paralelismo: *threads*

Exemplo: Paralelismo sem abstração no IBM 360



- ▶ Memória dividida em blocos de 2KB
- ▶ Associado a cada bloco, uma chave de 4 bits na CPU (proteção)
- ▶ Chave também para cada processo → Program Status Word (PSW)
- ▶ SO impede acesso a bloco sem a chave correta



Armazenamento nos Computadores

Gerenciamento de Memória sem abstração

Abstrações Simples de Memória

Gerenciamento de Memória Livre

Conclusão

Problema: Acesso Absoluto *versus* Relativo



0	16380
---	-------

⋮

ADD	28
MOV	24
	20
	16
	12
	8
	4
JMP 24	0

Problema: Acesso Absoluto *versus* Relativo



0	16380
---	-------

⋮

ADD	28
-----	----

MOV	24
-----	----

	20
--	----

	16
--	----

	12
--	----

	8
--	---

	4
--	---

JMP 24	0 ←
--------	-----

Problema: Acesso Absoluto *versus* Relativo



0	16380
---	-------

⋮

ADD	28
MOV	24 ←
	20
	16
	12
	8
	4
JMP 24	0

Problema: Acesso Absoluto *versus* Relativo



0	16380	0	16380
⋮		⋮	
ADD	28	CMP	28
MOV	24		24
	20		20
	16		16
	12		12
	8		8
	4		4
JMP 24	0	JMP 28	0

Problema: Acesso Absoluto *versus* Relativo



0	16380	0	16380
⋮		⋮	
ADD	28	CMP	28
MOV	24		24
	20		20
	16		16
	12		12
	8		8
	4		4
JMP 24	0	JMP 28	0 ←

Problema: Acesso Absoluto *versus* Relativo



0	16380	0	16380
⋮		⋮	
ADD	28	CMP	28
MOV	24		24
	20		20
	16		16
	12		12
	8		8
	4		4
JMP 24	0	JMP 28	0



Problema: Acesso Absoluto *versus* Relativo



0	16380	0	16380	0	32764
⋮		⋮		⋮	
ADD	28	CMP	28	CMP	16412
MOV	24		24		16408
	20		20		16404
	16		16		16400
	12		12		16396
	8		8		16392
	4		4		16388
JMP 24	0	JMP 28	0	JMP 28	16384
				0	16380
				⋮	
				ADD	28
				MOV	24
					20
					16
					12
					8
					4
				JMP 24	0

Problema: Acesso Absoluto *versus* Relativo



0	16380	0	16380	0	32764
⋮		⋮		⋮	
ADD	28	CMP	28	CMP	16412
MOV	24		24		16408
	20		20		16404
	16		16		16400
	12		12		16396
	8		8		16392
	4		4		16388
JMP 24	0	JMP 28	0	JMP 28	16384
				0	16380
				⋮	
				ADD	28
				MOV	24
					20
					16
					12
					8
					4
				JMP 24	0 ←

Problema: Acesso Absoluto *versus* Relativo



0	16380	0	16380	0	32764
⋮		⋮		⋮	
ADD	28	CMP	28	CMP	16412
MOV	24		24		16408
	20		20		16404
	16		16		16400
	12		12		16396
	8		8		16392
	4		4		16388
JMP 24	0	JMP 28	0	JMP 28	16384
				0	16380
				⋮	
				ADD	28
				MOV	24
					20
					16
					12
					8
					4
				JMP 24	0

Problema: Acesso Absoluto *versus* Relativo



0	16380	0	16380	0	32764
⋮		⋮		⋮	
ADD	28	CMP	28	CMP	16412
MOV	24		24		16408
	20		20		16404
	16		16		16400
	12		12		16396
	8		8		16392
	4		4		16388
JMP 24	0	JMP 28	0	JMP 28	16384 ←
				0	16380
				⋮	
				ADD	28
				MOV	24
					20
					16
					12
					8
					4
				JMP 24	0

Problema: Acesso Absoluto *versus* Relativo



0	16380	0	16380	0	32764
⋮		⋮		⋮	
ADD	28	CMP	28	CMP	16412
MOV	24		24		16408
	20		20		16404
	16		16		16400
	12		12		16396
	8		8		16392
	4		4		16388
JMP 24	0	JMP 28	0	JMP 28	16384
				0	16380
				⋮	
				ADD	28
				MOV	24
					20
					16
					12
					8
					4
				JMP 24	0

Problema: Acesso Absoluto *versus* Relativo



0	16380	0	16380	0	32764
⋮		⋮		⋮	
ADD	28	CMP	28	CMP	16412
MOV	24		24		16408
	20		20		16404
	16		16		16400
	12		12		16396
	8		8		16392
	4		4		16388
JMP 24	0	JMP 28	0	JMP 28	16384
				0	16380

Ambos fazem referência absoluta de memória

⋮	
ADD	28
MOV	24
	20
	16
	12
	8
	4
JMP 24	0

Problema: Acesso Absoluto *versus* Relativo



0	16380	0	16380	0	32764
⋮		⋮		⋮	
ADD	28	CMP	28	CMP	16412
MOV	24		24		16408
	20		20		16404
	16		16		16400
	12		12		16396
	8		8		16392
	4		4		16388
JMP 24	0	JMP 28	0	JMP 28	16384
				0	16380

Ambos fazem referência absoluta de memória

Solução: adicionar 16384 a todos os endereçamentos do segundo programa

⋮	
ADD	28
MOV	24
	20
	16
	12
	8
	4
JMP 24	0

Problema: Acesso Absoluto *versus* Relativo



0	16380	0	16380	0	32764
⋮		⋮		⋮	
ADD	28	CMP	28	CMP	16412
MOV	24		24		16408
	20		20		16404
	16		16		16400
	12		12		16396
	8		8		16392
	4		4		16388
JMP 24	0	JMP 28	0	JMP 28	16384
				0	16380

Ambos fazem referência absoluta de memória

Solução: adicionar 16384 a todos os endereçamentos do segundo programa

REALOCAÇÃO ESTÁTICA

⋮	
ADD	28
MOV	24
	20
	16
	12
	8
	4
JMP 24	0

Espaço de endereçamento do processo



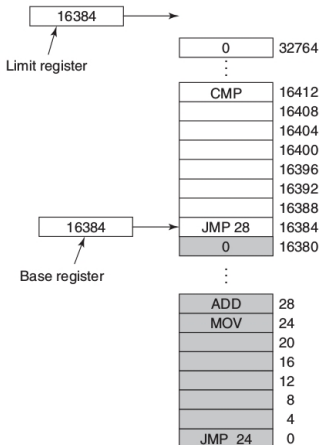
- ▶ Exposição da memória física aos processos
 - ▶ Endereçamento direto
 - ▶ Danos intencionais ou acidentais ao sistema
 - ▶ Difícil implementação de paralelismo
 - ▶ Problemático mesmo sem paralelismo
- ▶ Um espaço de endereçamento para cada processo
 - ▶ Solução de dois problemas: proteção e realocação
 - ▶ Conjunto de endereços que o processo pode usar
 - ▶ Cada processo tem o seu
- ▶ Exemplos:
 - ▶ Portas de E/S do Pentium: 0 a 16383
 - ▶ Endereços IPv4: 0 a $2^{32} - 1$

Registradores-base e registradores-limite



- ▶ Solução simples da **realocação dinâmica**
- ▶ Mapeia o **espaço de endereçamento** de cada processo em uma parte diferente da **memória física**
- ▶ Dois registradores de *hardware* especiais
 - ▶ Registradores-base
 - ▶ Registradores-limite
- ▶ Cada referência à memória tem acrescido o valor base
 - ▶ Executado pelo *hardware* da CPU
 - ▶ Também verifica se resultado é menor que limite

Registradores-base e registradores-limite



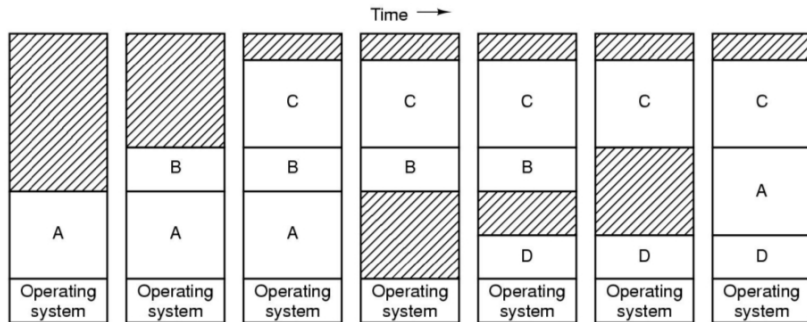
- ▶ Solução simples que dá a cada processo um espaço exclusivo
- ▶ Em alguns casos apenas o SO pode acessar os registradores
- ▶ Desvantagem: a cada acesso deve ser executada
 - ▶ Operação de adição
 - ▶ Operação de comparação
- ▶ Exemplos:
 - ▶ CDC6600
 - ▶ Intel 8088

Troca de Memória (*Swapping*)



- ▶ RAM necessária para todos os processos maior que a disponível
 - ▶ Problema recorrente!
- ▶ Estratégia mais simples: *Swapping*
 - ▶ Carregar processo para execução
 - ▶ Devolvê-lo ao disco quando inativo
- ▶ Muitas trocas podem deixar espaços vazios: **Fragmentação**
 - ▶ Compactação de memória
 - ▶ Aumenta espaços contínuos
 - ▶ Pouco utilizada por questões de desempenho

Exemplo de *swapping* de processos

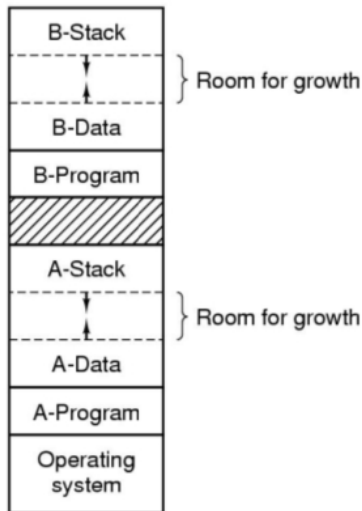
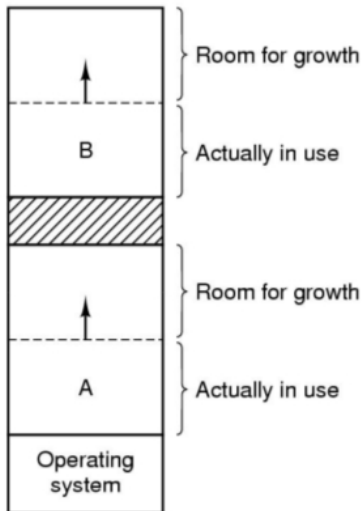


Alocação de memória no *swapping*



- ▶ Qual deve ser a quantidade de memória alocada a um processo?
- ▶ Processos com tamanho fixo → Solução simples
- ▶ Processos com alocação dinâmica
 - ▶ Simples havendo espaço livre adjacente
 - ▶ Caso contrário, deverá ocorrer realocação
 - ▶ Pode não ser possível → Suspensão ou término
- ▶ Pode-se alocar memória extra a priori
- ▶ Processos também podem ter duas áreas de expansão
 - ▶ Dados (heap): variáveis dinâmicas
 - ▶ Pilha: Variáveis locais comuns e endereços de retorno

Espaço de alocação no *swapping*)



Conteúdo



Armazenamento nos Computadores

Gerenciamento de Memória sem abstração

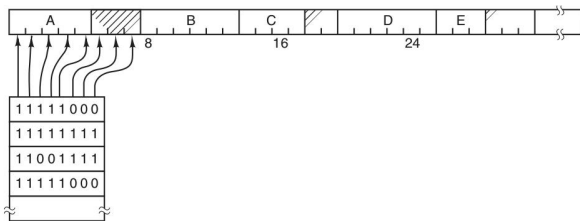
Abstrações Simples de Memória

Gerenciamento de Memória Livre

Conclusão



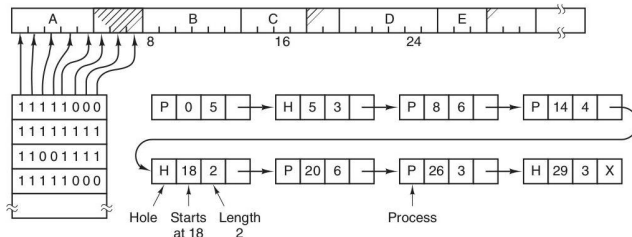
- ▶ Duas maneiras de verificar utilização de memória
 - ▶ Mapa de bits
 - ▶ Listas encadeadas
- ▶ Mapa de bits
 - ▶ Memória dividida em unidades de alocação
 - ▶ A cada unidade é atribuído um bit (0=livre/1=ocupado)
 - ▶ Tamanho da unidade: parâmetro de projeto
 - ▶ Alocação lenta: procura de sequência de unidades livres



Gerenciamento com listas encadeadas



- ▶ Lista de segmentos alocados e disponíveis
- ▶ Cada elemento da lista:
 - ▶ Especifica status (H=livre / P=ocupado com um processo)
 - ▶ Endereço de início
 - ▶ Comprimento
 - ▶ Ponteiro para próximo elemento



Vizinhos com listas encadeadas



- ▶ Lista ordenada permite atualização rápida
- ▶ Processo que termina execução tem dois vizinhos na lista
 - ▶ Segmentos alocados a um processo
 - ▶ Memória livre
- ▶ Existem 4 combinações de vizinhos





- ▶ Diversos algoritmos de alocação
 - ▶ *First fit*: rápido
 - ▶ *Next fit*: memoriza posição encontrada
 - ▶ *Best fit*: escolhe menor segmento (mais lento)
 - ▶ *Worst fit*: escolhe maior segmento (pior?)
- ▶ Listas separadas para **processos** e **espaços livres**
 - ▶ Alocação mais rápida
 - ▶ Maior complexidade
 - ▶ Liberação mais lenta: troca de listas
 - ▶ *First fit* e *Best fit* igualmente rápidos



Armazenamento nos Computadores

Gerenciamento de Memória sem abstração

Abstrações Simples de Memória

Gerenciamento de Memória Livre

Conclusão

Exercício: Gerenciamento de Memória Livre



- Considere um sistema de troca no qual a memória consiste nos seguintes tamanhos de lacunas na ordem da memória:

20 MB, 10 MB, 18 MB, 9 MB, 12 MB e 15 MB

Qual lacuna é pega para sucessivas solicitações de segmentos de para o algoritmos **First fit**, **best fit**, **worst fit** e **next fit**.

- (a) 6 MB
- (b) 10 MB
- (c) 12 MB
- (d) 9 MB

Dúvidas?



Obrigado pela atenção!